

Reading and Writing Data

readr and haven

2026-07-30

readr



Function	Reads
<code>read_csv()</code>	Comma separated values
<code>read_csv2()</code>	Semi-colon separate values
<code>read_delim()</code>	General delimited files
<code>read_fwf()</code>	Fixed width files
<code>read_log()</code>	Apache log files
<code>read_table()</code>	Space separated files
<code>read_tsv()</code>	Tab delimited values

Importing Data

```
1 dataset <- read_csv("file_name.csv")  
2 dataset
```

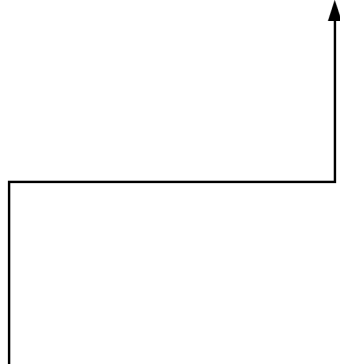
R functions

```
x <- f(arg = 1)
```

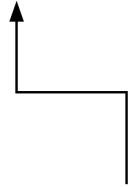
R functions

x <- f(arg = 1)

function
name

A diagram consisting of a vertical line extending upwards from the word 'function' and a horizontal line extending to the right from the top of that vertical line, ending with an arrowhead pointing to the orange 'f' in the code snippet above.

arguments

A diagram consisting of a vertical line extending upwards from the word 'arguments' and a horizontal line extending to the left from the top of that vertical line, ending with an arrowhead pointing to the opening parenthesis '(' in the code snippet above.

R functions



*this saves it in your
global environment*

x <- f(arg = 1)

assign
results of
f() to x

the name of
your results

Your Turn 1

Find `diabetes.csv` on your computer. Then read it into an object. Then view the results.

Your Turn 1

Find **diabetes.csv** on your computer. Then read it into an object. Then view the results.

```
1 diabetes <- read_csv("diabetes.csv")
```



new data alert!



diabetes

	id	chol	stab.glu	hdl	ratio	glyhb	location	age	gender	height	weight	frame	bp.
1	1000	203	82	56	3.6	4.31	Buckingham	46	female	62	121	medium	118
2	1001	165	97	24	6.9	4.44	Buckingham	29	female	64	218	large	112
3	1002	228	92	37	6.2	4.64	Buckingham	58	female	61	256	large	190
4	1003	78	93	12	6.5	4.63	Buckingham	67	male	67	119	large	110
5	1005	249	90	28	8.9	7.72	Buckingham	64	male	68	183	medium	138
6	1008	248	94	69	3.6	4.81	Buckingham	34	male	71	190	large	132
7	1011	195	92	41	4.8	4.84	Buckingham	30	male	69	191	medium	161
8	1015	227	75	44	5.2	3.94	Buckingham	37	male	59	170	medium	NA
9	1016	177	87	49	3.6	4.84	Buckingham	45	male	69	166	large	160
10	1022	263	89	40	6.6	5.78	Buckingham	55	female	63	202	small	108
11	1024	242	82	54	4.5	4.77	Louisa	60	female	65	156	medium	130
12	1029	215	128	34	6.3	4.97	Louisa	38	female	58	195	medium	102
13	1030	238	75	36	6.6	4.47	Louisa	27	female	60	170	medium	130
14	1031	183	79	46	4.0	4.59	Louisa	40	female	59	165	medium	NA
15	1035	191	76	30	6.4	4.67	Louisa	36	male	69	183	medium	100
16	1036	213	83	47	4.5	3.41	Louisa	33	female	65	157	medium	130
17	1037	255	78	38	6.7	4.33	Louisa	50	female	65	183	medium	130
18	1041	220	112	64	3.6	4.52	Louisa	20	male	67	150	medium	100

Where does it come from?
diabetes.csv (etc)
study: diabetes in
African Americans

How can I use it?

```
diabetes <-  
readr::read_csv("diabetes.csv")  
View(diabetes)
```



*this saves it in your
global environment*

```
1 diabetes
```

```
# A tibble: 403 × 19
```

	id	chol	stab.glu	hdl	ratio	glyhgb	location	age
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<chr>	<dbl>
1	1000	203	82	56	3.60	4.31	Buckingham	46
2	1001	165	97	24	6.90	4.44	Buckingham	29
3	1002	228	92	37	6.20	4.64	Buckingham	58
4	1003	78	93	12	6.5	4.63	Buckingham	67
5	1005	249	90	28	8.90	7.72	Buckingham	64
6	1008	248	94	69	3.60	4.81	Buckingham	34
7	1011	195	92	41	4.80	4.84	Buckingham	30
8	1015	227	75	44	5.20	3.94	Buckingham	37
9	1016	177	87	49	3.60	4.84	Buckingham	45
10	1022	263	89	40	6.60	5.78	Buckingham	55
"	...	...	...	...	...	...	...	...

Tibbles

`data.frames` are the basic form of rectangular data in R (columns of variables, rows of observations)

`read_csv()` reads the data into a `tibble`, a modern version of the data frame.

a `tibble` is a data frame

Missing values

It's common to use codes for missing values (-99, 9999)

The na option can change these values to NA

```
1 read_csv(  
2   "a,b,c,d  
3   1,-99,3,4  
4   5,6,-99,8",  
5   na = "-99"  
6 )
```

A tibble: 2 × 4

	a	b	c	d
	<dbl>	<dbl>	<dbl>	<dbl>
1	1	NA	3	4
2	5	6	NA	8

Parsing data types

The read functions in readr try to *guess* each data type, but sometimes it's *wrong*

To tell readr how to parse the columns, add the argument **col_types** to `read_csv()`

```
1 diabetes <- read_csv(  
2   "diabetes.csv",  
3   col_types = list(id = col_character())  
4 )
```

Parsing data types

Or use a string for
each variable type:

```
col_type = "cci"
```


Parsing data types

Your Turn 2

Set the 4 column types to be: integer, double, character, and unknown (guess)

```
1 read_csv(  
2     "a,b,c,d  
3     1,2,3,4  
4     5,6,7,8",  
5     col_types = ""  
6 )
```

Your Turn 2

Set the 4 column types to be: integer, double, character, and unknown (guess)

```
1 read_csv(  
2   "a,b,c,d  
3   1,2,3,4  
4   5,6,7,8",  
5   col_types = "idc?"  
6 )
```

```
# A tibble: 2 × 4  
      a      b c      d  
  <int> <dbl> <chr> <dbl>  
1     1     2 3     4  
2     5     6 7     8
```



Function	Software
<code>read_sas()</code>	SAS
<code>read_xpt()</code>	SAS
<code>read_spss()</code>	SPSS
<code>read_sav()</code>	SPSS
<code>read_por()</code>	SPSS
<code>read_stata()</code>	Stata
<code>read_dta()</code>	Stata

Heads up!

haven is *not* a core member of the tidyverse.
That means you need to load it with
`library(haven)`.

Your Turn 3

There are several versions of the diabetes file besides CSV. Pick a file format you or your colleagues use and import them using the corresponding function from haven.

Your Turn 3

```
1 library(haven)
2 diabetes <- read_sas("diabetes.sas7bdat")
```

Your Turn 3

```
1 diabetes
```

```
# A tibble: 403 × 19
```

	id	chol	stab_glu	hdl	ratio	glyhb	location	age
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<chr>	<dbl>
1	1000	203	82	56	3.60	4.31	Buckingham	46
2	1001	165	97	24	6.90	4.44	Buckingham	29
3	1002	228	92	37	6.20	4.64	Buckingham	58
4	1003	78	93	12	6.5	4.63	Buckingham	67
5	1005	249	90	28	8.90	7.72	Buckingham	64
6	1008	248	94	69	3.60	4.81	Buckingham	34
7	1011	195	92	41	4.80	4.84	Buckingham	30
8	1015	227	75	44	5.20	3.94	Buckingham	37
9	1016	177	87	49	3.60	4.84	Buckingham	45
10	1022	263	89	40	6.60	5.78	Buckingham	55

```
# A tibble: 403 × 19
```


Writing data

Function	Writes
<code>write_csv()</code>	Comma separated values
<code>write_excel_csv()</code>	CSV that you plan to open in Excel
<code>write_delim()</code>	General delimited files
<code>write_file()</code>	A single string, written as is
<code>write_lines()</code>	A vector of strings, one string per line
<code>write_tsv()</code>	Tab delimited values
<code>write_rds()</code>	A data type used by R to save objects
<code>write_xpt()</code>	SAS transport format, <code>.xpt</code>
<code>write_sas()</code>	SAS <code>.sas7bdat</code> files (experimental)

Function	Writes
<code>write_sav()</code>	SPSS <code>.sav</code> files
<code>write_stata()</code>	Stata <code>.dta</code> files

Writing data

```
1 write_csv(diabetes, file = "diabetes-clean.csv")
```

Your Turn 4

R has a few data file types, such as RDS and .Rdata. Save `diabetes` as "`diabetes.Rds`".

Your Turn 4

R has a few data file types, such as RDS and .Rdata. Save `diabetes` as "`diabetes.Rds`".

```
1 write_rds(diabetes, "diabetes.Rds")
```